Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- **HTTP Client Message Handler in Web API**
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

HTTP Client Message Handler in Web API

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

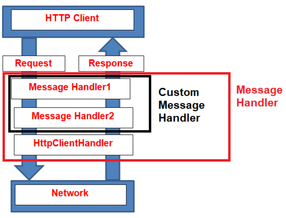
HTTP Client Message Handler in Web API with Real-Time examples

In this article, I am going to discuss **HTTP Client Message Handler in Web API** with real-time examples. As we already discussed in **HTTP Message Handler Article** that a **Message Handler** is a class that receives an **HTTP request** and returns an **HTTP response**. The **Message handler** is derived from the abstract **HttpMessageHandler** class. There are two types of HTTP Message Handlers as follows

1. **The Server Side HTTP Message Handlers** – we already discussed
2. Client Side HTTP Message Handlers – will discuss in this article

HTTP Client Message Handlers in Web API

The **HttpClient** class uses a message handler to process the requests on the client side. The default handler provided by the dot net framework is **HttpClientHandler.** This **HTTP Client Message Handler** sends the request over the network and also gets the response from the server. As a developer if you want, then you can also create your own custom message handlers and then insert the **custom message handlers** into the pipeline in the client side as shown in the below image.



Creating a Custom HTTP Client Message Handler:

Let us discuss how to create a Custom HTTP Client Message Handler. To create a custom HTTP Client message handler, what we need to do is, we need to create a custom class and that class should be derived from the **System.Net.Http.DelegatingHandler** class. Then the class should override the **SendAsync** method. The signature of the **SendAsync** method as following:

```
protected async override Task<HttpResponseMessage> SendAsync(
    HttpRequestMessage request, CancellationToken cancellationToken)
```

The **SendAsync** method takes an **HttpRequestMessage** as input and asynchronously returns an **HttpResponseMessage**. A typical implementation does the following:

1. Process the request message.
2. Call the base.SendAsync method to send the request to the inner handler.
3. The inner handler returns a response message. (This step is asynchronous.)
4. Process the response message and returns the response to the caller.

Creating a Custom Message Handler:

The following example shows the creation of a custom message handler which adds a custom header to the outgoing request:

```
using System.Net.Http;
using System.Threading.Tasks;

namespace ClientSideMessageHandler.Models
{
    class MessageHandler1 : DelegatingHandler
    {
        private int _count = 0;
        protected override Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, System.Threading.CancellationToken
cancellationToken)
        {
            System.Threading.Interlocked.Increment(ref _count);
            request.Headers.Add("X-Custom-Header", _count.ToString());
            return base.SendAsync(request, cancellationToken);
        }
    }
}
```

The call to the **base.SendAsync** method is asynchronous. If your handler going to do some work after this call, then use the **await** keyword to resume execution after the method completes.

The following example shows a handler that logs error codes. The example shows how to get at the response inside the handler.

```
using System.IO;
using System.Net.Http;
using System.Threading.Tasks;

namespace ClientSideMessageHandler.Models
{
    class LoggingHandler : DelegatingHandler
    {
        StreamWriter _writer;

        public LoggingHandler(Stream stream)
        {
            _writer = new StreamWriter(stream);
        }

        protected override async Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, System.Threading.CancellationToken
cancellationToken)
        {
            var response = await base.SendAsync(request, cancellationToken);

            if (!response.IsSuccessStatusCode)
            {
                _writer.WriteLine("{0}\t{1}\t{2}", request.RequestUri,
                    (int)response.StatusCode, response.Headers.Date);
            }
            return response;
        }

        protected override void Dispose(bool disposing)
        {
            if (disposing)
            {
                _writer.Dispose();
            }
            base.Dispose(disposing);
        }
    }
}
```

Adding Message Handlers to the Client Pipeline

To add a custom message handlers to **HttpClient** pipeline, we need to use the **HttpClientFactory.Create** method as shown below.

```
HttpClient client = HttpClientFactory.Create(new MessageHandler1(), new MessageHandler2());
```

The Message handlers are called in the order that we pass them into the **Create** method of the **HttpClientFactory** class. The reason is handlers are nested the response message travels in the other direction. That is, the last handler is the first to get the response message.

In the next article, I am going to discuss How to Implement the **Token Based Authentication in ASP.NET Web API**. Here, in this article, I try to explain **HTTP Client Message Handler in Web API** with some examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.

DOT NET TUTORIALS
Dot Net Tutorials
About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

3 thoughts on "HTTP Client Message Handler in Web API"

DUONG THANH NAM
SEPTEMBER 12, 2019 AT 2:31 PM

Hey Guy! Please Add Button Next Lesson and Previou Lesson

Reply

JAMES
NOVEMBER 1, 2019 AT 12:13 PM

At the end of the article, then next article link is provided and at the start previous article link provided.

Reply

FUTURE
FEBRUARY 21, 2021 AT 8:12 AM

Easy and simple approach which helps starter to learn all the basic in very short period. Nice approach, I really appreciate you

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* | Email* | Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel